

Explicacao das Funcoes do Projeto TechStore Plus

Este documento detalha a logica por tras do projeto de chatbot de atendimento da loja ficticia TechStore Plus. O projeto tem duas versoes principais: uma versao inicial usando a API da OpenAI diretamente e uma versao refatorada com LangChain LCEL, Pydantic e uma arquitetura em cadeia.

Visao Geral

O fluxo geral e: mensagem do cliente -> analise/classificacao da intencao -> geracao da resposta personalizada -> criacao de resumo estruturado -> salvamento em JSON -> consolidacao dos atendimentos.

A ideia central e separar tres responsabilidades: entender o problema do cliente, responder de forma adequada ao caso e registrar o atendimento de forma estruturada.

Notebook Semana 1: OpenAI Direto

Arquivo: TechStorePlus_Customer_Service_Chatbot_Project.ipynb. Esta versao usa funcoes Python tradicionais, chamadas diretas ao modelo e parsing manual de JSON.

ConversationSession

Classe responsavel por guardar o estado de uma conversa. Ela mantem o ID do cliente, o horario de criacao e uma lista de mensagens com os papeis system, user e assistant.

__init__(customer_id=None)

Cria uma nova sessao. Se nenhum customer_id for informado, gera um ID automatico no formato CUST-XXXXXXX. Tambem inicializa o historico com a mensagem system, que contem o comportamento esperado do chatbot e o contexto da empresa.

add_user_message(message)

Adiciona ao historico uma mensagem enviada pelo cliente. Isso permite que as proximas chamadas ao modelo tenham contexto do que ja foi dito.

add_assistant_message(message)

Adiciona ao historico uma resposta do chatbot. Assim, a conversa fica completa e pode ser reutilizada em respostas futuras ou em resumos.

get_public_history()

Retorna o historico sem a mensagem system. A mensagem system e uma instrucao interna para o modelo e nao deve aparecer em resumos ou registros externos.

analyze_customer_query(query)

Analisa a mensagem do cliente com o modelo gpt-4o-mini. A funcao pede que o modelo retorne apenas JSON valido contendo sentimento, emocoes, categoria, urgencia, produtos mencionados, informacoes extraidas, roteamento recomendado e um resumo do raciocinio.

A temperatura e 0 porque classificacao deve ser previsivel. Depois da chamada, a funcao usa json.loads para converter a resposta em dicionario Python. Se o modelo devolver JSON dentro de bloco Markdown, a funcao remove as crases e tenta converter novamente.

generate_personalized_response(session, user_query, analysis)

Gera a resposta final para o cliente usando o historico da sessao, a mensagem do usuario, a analise estruturada e as politicas da TechStore Plus. Primeiro adiciona a mensagem do cliente ao historico, depois envia o historico completo mais uma instrucao de resposta ao modelo.

A temperatura e 0.4 para permitir uma resposta mais natural, mas ainda controlada. Depois de receber o texto, a funcao salva a resposta no historico da sessao e retorna esse texto.

chatbot_reply(session, user_query)

Funcao principal da Semana 1. Ela orquestra duas etapas: primeiro chama `analyze_customer_query` para entender a mensagem; depois chama `generate_personalized_response` para responder. Retorna `customer_id`, `query`, `analysis` e `reply`.

generate_conversation_summary(...)

Cria um resumo estruturado da conversa. Na versao com OpenAI, pega o historico publico da sessao, pede ao modelo para resumir em um paragrafo e monta um JSON com timestamp, `customer_id`, categoria, sentimento, urgencia, produtos, informacoes extraidas, status de resolucao, acoes tomadas e follow-up.

save_conversation_json(conversation_summary)

Salva o resumo da conversa em `conversation_data` com nome no formato `conversation_{customer_id}_{timestamp}.json`. Retorna o caminho do arquivo salvo.

consolidate_conversations(output_file='consolidated_conversations.json')

Le todos os arquivos `conversation_*.json` da pasta `conversation_data`, junta os registros em uma lista e salva um arquivo consolidado com `generated_at`, `total_conversations` e `conversations`.

Modo Mock da Semana 1

O modo mock e ativado com `MOCK_MODE = True`. Ele substitui funcoes que chamariam a API por implementacoes locais baseadas em regras, permitindo demonstrar o projeto sem gastar tokens.

mock_extract_products(query)

Procura nomes de produtos com base em palavras-chave. Por exemplo, se a frase contem `iphone 15`, adiciona `iPhone 15` a lista de produtos mencionados.

mock_extract_information(query)

Usa expressoes regulares para extrair numero de pedido, valores em dolar e datas em ingles. Retorna um dicionario com `order_number`, `purchase_date` e `amount`.

analyze_customer_query(query) no mock

Classifica urgencia, sentimento, emocoes, categoria e roteamento usando regras. Termos como `emergency`, `urgent` e `tomorrow` levam a urgencia `high`. Termos como `thank` e `excellent` indicam sentimento positivo. Termos como `never arrived` e `doesn't work` indicam sentimento negativo.

A categoria tambem vem de palavras-chave: `receipt` e `payment` viram `billing`; `return` e `refund` viram `return`; `warranty` vira `warranty`; `configure` e `doesn't work` viram `technical`; `stock`, `shipping`, `recommend` e `budget` viram `product_information`. Se a urgencia for `high`, o roteamento recomendado vira `Priority Support Team`.

generate_personalized_response(...) no mock

Monta uma resposta com blocos fixos: abertura conforme urgencia ou sentimento, acao conforme categoria, detalhes de pedido/produto, roteamento recomendado e observacao de prioridade quando o caso e urgente.

Tambem atualiza o historico da sessao.

generate_conversation_summary(...) no mock

Cria o JSON final sem chamar o modelo. Usa a ultima mensagem do cliente, a categoria, o sentimento e a urgencia para montar uma frase de resumo local.

mock_analyze_customer_query(query)

Versao mock mais simples, usada como fallback/demonstracao. Classifica urgencia, sentimento e categoria, mas nao extrai produtos e informacoes com o mesmo nivel da versao mock principal.

Notebook Semana 2: LangChain LCEL

Arquivo: TechStorePlus_LangChain_LCEL_Chatbot.ipynb. Esta versao organiza o projeto como uma cadeia modular usando LangChain LCEL e modelos Pydantic.

ExtractedEntities

Modelo Pydantic que define as entidades extraidas da consulta: product_name, order_number e date. Isso torna a saida da analise mais previsivel.

QueryAnalysis

Modelo Pydantic que define a estrutura da analise: query_category, urgency_level, customer_sentiment e entities. O uso de Literal limita os valores aceitos e impede categorias inventadas pelo modelo.

analysis_chain

Componente LCEL equivalente a analyze_customer_query. Ele conecta o prompt de analise ao modelo com structured output: analysis_prompt | analysis_llm.with_structured_output(QueryAnalysis). Assim, a resposta precisa seguir o schema Pydantic.

CATEGORY_PROMPTS

Dicionario com prompts especificos por categoria: technical_support, billing, returns, product_inquiry e general_information. Cada categoria muda a persona e as instrucoes do atendente.

route_response(inputs)

Funcao de roteamento dinamico. Ela recebe a query e a analise, le analysis.query_category, escolhe o prompt correto em CATEGORY_PROMPTS, monta uma mini-chain prompt | response_llm e invoca essa chain com query, sentimento, urgencia, entidades e contexto da empresa.

ConversationSummary

Modelo Pydantic para o resultado final do atendimento. Define timestamp, customer_id, conversation_summary, query_category, customer_sentiment, urgency_level, mentioned_products, extracted_information, resolution_status, actions_taken e follow_up_required.

_response_text(response)

Extrai texto legivel de uma resposta LangChain. Se a resposta tiver atributo content, usa esse atributo. Se o conteudo for lista, junta as partes em uma string. Caso contrario, converte para string.

_short_action_from_response(response)

Cria uma versao curta da resposta para registrar em actions_taken. Remove quebras de linha, limita a 180 caracteres e adiciona retencencias quando necessario.

build_summary(inputs)

Monta o ConversationSummary a partir da análise e da resposta. Não chama LLM. Extrai produto, pedido e data; mapeia urgência para status; cria uma frase de resumo; monta actions_taken; define follow_up_required como verdadeiro para urgência high ou medium.

O mapeamento de status é: high -> escalated, medium -> pending e low -> resolved. Essa regra é lógica de negócio, não uma decisão livre do modelo.

save_conversation_json(summary)

Salva o ConversationSummary em JSON dentro de conversation_data. Na Semana 2, os arquivos começam com week2_conversation_, separando-os dos arquivos da Semana 1.

consolidate_conversations(output_file='week2_consolidated_conversations.json')

Consolida apenas os arquivos week2_conversation_*.json, criando um arquivo final com generated_at, total_conversations e conversations.

chain_with_context

Cadeia LCEL completa que preserva informações intermediárias. Primeiro adiciona analysis, depois response e depois summary. Isso permite visualizar análise, resposta e resumo na mesma execução.

full_chain

Versão final da cadeia que retorna apenas x['summary']. Ela existe porque a saída esperada formalmente é uma instância de ConversationSummary.

starter.ipynb

O arquivo starter.ipynb tem uma função simples chamada greet(name), usada como exemplo inicial. Ela não participa da arquitetura principal do chatbot.

Resumo Final

A Semana 1 mostra o chatbot funcionando de forma direta: funções Python -> OpenAI API -> JSON manual -> resposta -> arquivo. A Semana 2 melhora a arquitetura: LCEL chain -> análise validada -> roteamento por categoria -> resumo estruturado.

A diferença mais importante é que a Semana 2 separa melhor as responsabilidades. A análise vira um objeto validado, o roteamento fica centralizado em route_response() e o resumo final é criado por regra em build_summary(), sem depender de mais uma resposta do modelo.